Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 2

«Компилятор, компоновщик и динамические библиотеки»

по дисциплине «Технологии разработки современного программного обеспечения»

Выполнил
студент гр. 5131001/30501 Чупров Е. А.

<подпись>

Проверил
ассистент Завадский Е. В.

<подпись>

Санкт-Петербург
2026

СОДЕРЖАНИЕ

1 Формулировка задания **3**

2 Теоретические сведения **4**

3 Результаты работы **5**

3.1 Структура проекта 5

3.2 Программа формирования PNG 5

3.3 Статическая версия 6

3.4 Динамическая версия 6

3.5 Blob-версия 7

3.6 Загрузчик ELF 8

3.7 Корневой Makefile и инкрементная сборка 9

3.8 Проверка результатов 10

4 Выводы **11**

1 ФОРМУЛИРОВКА ЗАДАНИЯ

Целью работы – изучение этапов сборки программы с учетом особенностей статической и динамической компоновки и практическое освоение средств автоматизации данного процесса.

В рамках выполнения лабораторной работы необходимо решить следующие задачи:

1. Изучить синтаксис файла конфигурации (Makefile) системы сборки make.

2. Разработать нарисованным программу, в нем которая формирует текстовым сообщением PNG-файла с помощью с шрифта, загруженного из файла *.ttf с использованием библиотеки zlib, libpng, freetype.

3. Разработать Makefile для сборки трех версий программы:

_со статической компоновкой трех библиотек;

_с динамическая загрузка всех трех библиотек в начале

_в виде blob-файла, загружаемого с диска при помощи загрузчика elf-loader

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Сборка программы и Makefile

Программа на C собирается в четыре шага. Препроцессор подставляет заголовочные файлы и раскрывает макросы, в том числе ветки #ifdef. Компилятор переводит текст в ассемблер, ассемблер получает объектный файл .o. Компоновщик соединяет объектные файлы и подставляет адреса функций.

Команды сборки записаны в Makefile. У правила есть цель, список зависимостей и команды. Make сравнивает время изменения файлов. Если цель новее всех зависимостей, команды не запускаются. Если изменился один исходник, заново собирается только то, что от него зависит. Так получается инкрементная сборка.

В программе три режима сборки выбираются макросом. Один и тот же main.c компилируется с -DMODE_STATIC, -DMODE_DYNAMIC или -DMODE_BLOB. Ветки #ifdef оставляют в файле нужную точку входа и нужный способ подключения библиотек.

2.2 Статическая и динамическая компоновка

Статическая библиотека — это архив .a из уже скомпилированных объектных файлов. Компоновщик берёт из архива те .o, в которых есть нужные функции, и кладёт их в программу. После сборки код библиотеки лежит внутри файла. При запуске эту библиотеку искать не нужно. Файл получается больше, а новую версию библиотеки можно подключить только новой сборкой.

Разделяемая библиотека — отдельный файл .so. В программу попадает не её код, а имя библиотеки и имена функций. При запуске динамический загрузчик находит .so, загружает его в память и подставляет настоящие адреса. Один файл .so могут использовать несколько программ, поэтому сама программа меньше.

Библиотеку .so собирают с флагом -fPIC: код не привязан к одному адресу и может быть загружен куда угодно. У библиотеки есть короткое внутреннее имя, SONAME. У zlib это libz.so.1. По нему загрузчик понимает, что перед ним нужная версия, даже если файл лежит рядом с программой, а не в системном каталоге.

Список библиотек, которые программа просит загрузить при старте, хранится в ELF в записях DT_NEEDED. Динамическая версия этой работы так не делает. В начале main она открывает три библиотеки сама. dlopen загружает файл, dlsym возвращает адрес нужной функции, dlclose освобождает библиотеку.

2.3 Формат ELF и blob-файл

Исполняемый файл в Linux имеет формат ELF. В заголовке записаны тип файла, архитектура и адрес точки входа e_entry. Таблица программных заголовков описывает, какие части файла положить в память. Код и данные лежат в сегментах PT_LOAD. Обычная программа содержит сегмент PT_INTERP. Ядро по нему запускает динамический загрузчик, а тот читает DT_NEEDED и подгружает библиотеки. У blob-файла нет ни PT_INTERP, ни записей DT_NEEDED. Командой ./app-blob его не запустить: для него написан отдельный загрузчик.

Blob собран как PIE, флагами -fPIE и -pie. Файл можно положить в память по любому адресу. Места, где внутри файла записан абсолютный адрес, описаны поправками R_X86_64_RELATIVE. Загрузчик прибавляет к такому адресу базу, по которой файл оказался в памяти. Других поправок в файле нет: все внешние имена уже разрешены при сборке, импортировать символы не нужно. Своей стандартной библиотеки у blob нет. Его собирают с -nostdlib. Вызовы fopen, malloc, printf и остальных функций libc идут через таблицу указателей. Загрузчик заполняет таблицу адресами функций своего процесса и передаёт её в точку входа вместе с тремя аргументами программы: путём к шрифту, именем PNG и текстом. Внутри blob остаются короткие обёртки с обычными именами. Обёртка просто переходит

по указателю из таблицы. Поэтому в blob можно вшить статические zlib, libpng и FreeType и не связывать их с libc.

Загрузчик построен на библиотеке elfload. Она читает заголовок ELF, копирует сегменты PT_LOAD в подготовленную память и применяет поправки R_X86_64_RELATIVE. После этого вызывается точка входа blob_entry. Её адрес берётся из поля e_entry, а не поиском символа в таблице экспорта. Снаружи символ не виден.

2.4 Библиотеки zlib, libpng и FreeType

zlib сжимает данные алгоритмом DEFLATE. В PNG пиксели хранятся сжатыми, поэтому libpng вызывает zlib при записи файла. Программа дополнительно печатает строку zlibVersion. Так видно, что zlib вызвана напрямую, а не только изнутри libpng.

libpng создаёт структуры PNG, задаёт ширину, высоту и формат цвета и записывает строки изображения. В работе картинка цветная, по 8 бит на красный, зелёный и синий, без отдельного альфа-канала.

FreeType читает файл TTF, ставит размер символа и возвращает рисунок одной буквы. Программа переносит этот рисунок в буфер изображения. Чем сильнее буква покрывает пиксель, тем темнее пиксель. Фон остаётся белым. Размер символа — 48 пикселей, поля вокруг текста — 8 пикселей.

В проекте собраны zlib 1.2.11, libpng 1.6.37 и FreeType 2.10.0.

3 РЕЗУЛЬТАТЫ РАБОТЫ

3.1 Структура проекта

В корне лежит Makefile. Он сам ничего не компилирует, а по очереди вызывает Makefile в каталогах библиотек, программы и загрузчика. Исходники программы находятся в app, загрузчик — в elfloader. Тексты библиотек лежат в zlib, libpng и freetype, и у каждой библиотеки свой Makefile.

Каталог build появляется при сборке. В нём готовые .a, .so и исполняемые файлы. Объектные файлы остаются рядом с исходниками, в каталогах object_files. Отдельной установки в префикс нет: свои Makefile сразу кладут библиотеки в build.

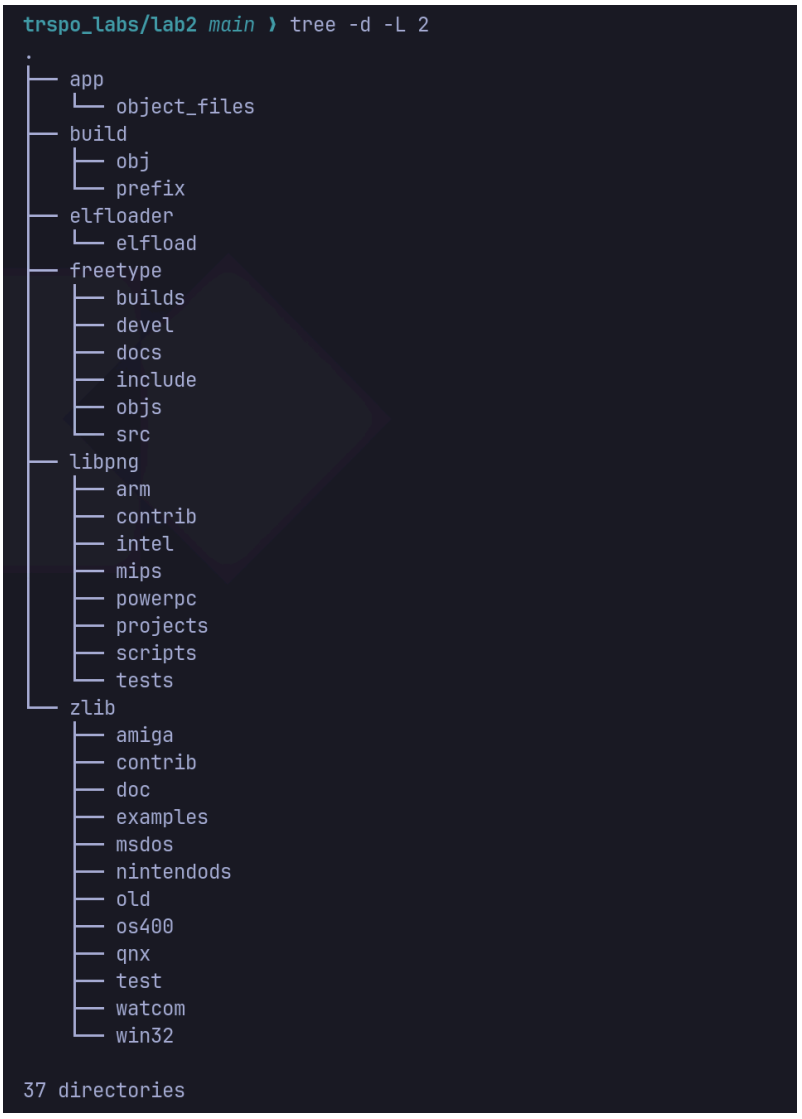


Рисунок 1 – Структура каталогов проекта

Таблица 1 — Назначение каталогов и файлов проекта

Элемент	**Назначение**
Makefile	Цели all, static, dynamic, blob, clean
app/	Исходные тексты программы и её Makefile
app/main.c	Точка входа main и точка входа blob_entry
app/funcs.c	Загрузка шрифта, отрисовка текста и запись PNG
app/dynload.c	Загрузка разделяемых библиотек через dlopen и dlsym
app/host_api.h	Описание структуры параметров blob и таблицы функций libc
app/host_shim.c	Обёртки функций libc для blob-версии
app/setjmp_stub.S	Переход к setjmp и longjmp загрузчика
app/blob.map	Скрытие всех символов blob-файла
elfloader/	Обёртка загрузчика и исходники elfload
zlib/	Исходные тексты zlib и Makefile
libpng/	Исходные тексты libpng и Makefile
freetype/	Исходные тексты FreeType и Makefile
build/	Готовые библиотеки и исполняемые файлы
font.ttf	Шрифт, используемый программой

3.2 Программа формирования PNG

Программа принимает три аргумента: файл шрифта, имя PNG и текст. Версии static и dynamic входят через main. Версия blob входит через blob_entry. В начале замеряется время, затем печатаются три строки: load, версия zlib и total.

load — время до начала рисунка. У dynamic в него входят dlopen и dlsym. У static и blob библиотеки уже лежат в файле, поэтому load около нуля. total — время от входа в программу до конца рисунка. У blob загрузка файла в память происходит до вызова blob_entry и в total не входит.

Если аргументов не три, программа печатает подсказку usage: ... font png text и возвращает 0. Если шрифт не открылся, PNG не удалось создать или библиотека не загрузилась, программа печатает сообщение и тоже возвращает 0. Аварийного завершения при плохих аргументах нет.

Дальше render_png инициализирует FreeType, открывает TTF и ставит размер 48 пикселей. Ширина картинки складывается из полей и ширины букв, высота — из метрик шрифта и тех же полей. Буфер заполняется белым, глифы рисуются серым по покрытию. libpng записывает буфер в файл, сжатие выполняет zlib.

Для текста TRSPO и шрифта font.ttf получается изображение 202×106 пикселей.



Рисунок 2 – Результат работы программы

3.3 Статическая версия

build/app-static связывается с тремя архивами:

```
gcc -o build/app-static \
    app/object_files/static/main.o app/object_files/static/funcs.o \
    build/libfreetype.a build/libpng.a build/libz.a -lm
```

В команде указаны файлы .a, а не .so. Компоновщик включает в программу только те объектные файлы библиотек, которые нужны. Код FreeType, libpng и zlib оказывается внутри app-static.

ldd build/app-static показывает libc и libm и не показывает libz, libpng и libfreetype. libc остаётся динамической: задание просит статически связать три библиотеки, а не весь glibc. Размер файла — 1 006 584 байта. load около нуля, потому что к входу в main три библиотеки уже часть программы.

3.4 Динамическая версия

build/app-dynamic связывается только с -ldl. zlib, libpng и FreeType в команду не входят:

```
gcc -o build/app-dynamic \
    app/object_files/dynamic/main.o app/object_files/dynamic/funcs.o \
    app/object_files/dynamic/dynload.o -ldl
```

В начале main функция load_libraries открывает libz.so.1, затем libpng.so, затем libfreetype.so. Файлы ищутся в каталоге самого исполняемого файла: путь читается из /proc/self/exe. Флаги — RTLD_NOW | RTLD_GLOBAL. Адреса FT_*, png_* и zlibVersion берутся через dlsym.

В dynload.h эти указатели подменяют обычные имена директивами #define. Файл funcs.c одинаковый для static и dynamic: в тексте вызываются FT_New_Face и png_write_image, а в динамической сборке имена раскрываются в обращения к указателям.

ldd build/app-dynamic показывает только libc. Отдельной libdl в этой системе нет, функции dlopen входят в libc. При этом readelf -d для libpng.so и libfreetype.so показывает зависимость от libz.so.1. Размер app-dynamic — 21 952 байта. load больше нуля: в него входит открытие трёх библиотек.

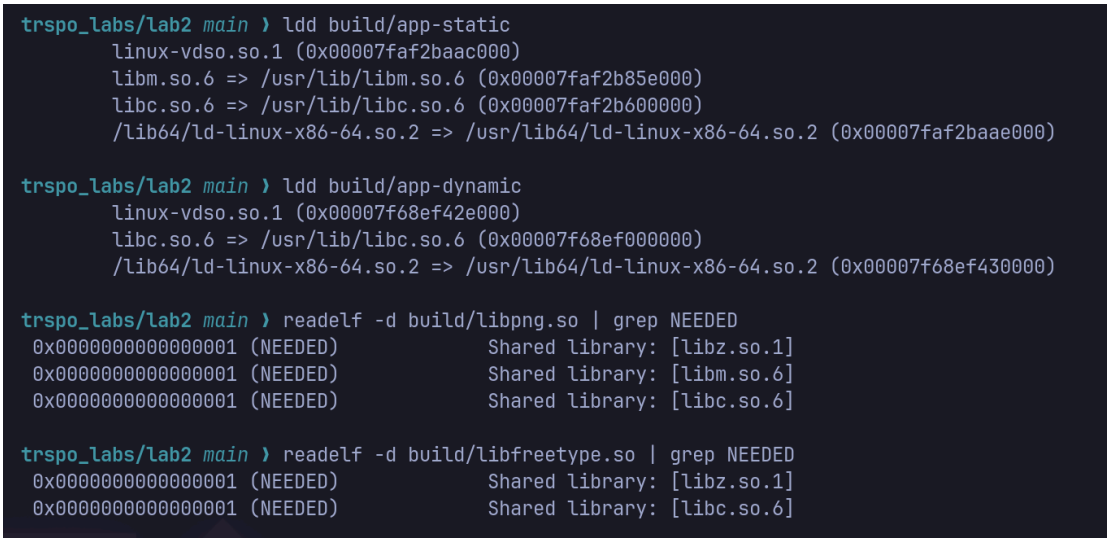


Рисунок 3 – Проверка зависимостей статической и динамической версий

3.5 Blob-версия

build/app-blob — PIE без стандартной библиотеки. Три архива вшиты в файл так же, как в static. Команда сборки:

```
gcc -nostdlib -pie -fPIE \
  -Wl,-e,blob_entry -Wl,--no-dynamic-linker -Wl,--no-undefined \
  -Wl,-Bsymbolic -Wl,--version-script,app/blob.map \
  -o build/app-blob \
  app/object_files/blob/main.o app/object_files/blob/funcs.o \
  app/object_files/blob/host_shim.o app/object_files/blob/setjmp_stub.o \
  build/libfreetype.a build/libpng.a build/libz.a -lgcc
```

-nostdlib отключает libc и стартовые файлы. -pie и -fPIE дают перемещаемый код. -Wl,-e,blob_entry записывает точку входа. --no-dynamic-linker не добавляет обычный загрузчик. blob.map скрывает все символы, поэтому таблицы экспорта нет. -lgcc подключает вспомогательные функции компилятора, не libc. readelf показывает тип DYN и точку входа 0x8040. В динамической секции нет DT_NEEDED. Символ blob_entry локальный, его адрес совпадает с e_entry.

Сегмента PT_INTERP нет. Все 1311 поправок имеют тип R_X86_64_RELATIVE. Размер файла — 1 004 584 байта.

Обращения к libc заменены обёртками из host_shim.c. Каждая обёртка вызывает функцию из таблицы host_api. FreeType в этой сборке собран с системным слоем src/base/ftsystem.c и открывает шрифт через fopen, а не через open и mmap. libpng сообщает об ошибке через setjmp и longjmp. В setjmp_stub.S стоит не вызов, а переход на setjmp загрузчика. Иначе longjmp вернулся бы в обёртку, а не в код записи PNG.

```
trspo_labs/lab2 main ❯ file build/app-blob
build/app-blob: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), static-pie linked, Buil
dID[sha1]=d86e17fdea1e170a6636658157fd350859695982, not stripped

trspo_labs/lab2 main ❯ readelf -h build/app-blob | head -20
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              DYN (Position-Independent Executable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x8040
  Start of program headers:          64 (bytes into file)
  Start of section headers:          1007528 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:         11
  Size of section headers:           64 (bytes)
  Number of section headers:         20
  Section header string table index: 19

trspo_labs/lab2 main ❯ readelf -d build/app-blob

Dynamic section at offset 0xdeed0 contains 12 entries:
  Tag        Type                         Name/Value
 0x000000006ffffef5 (GNU_HASH)           0x2d0
 0x0000000000000005 (STRTAB)             0x308
 0x0000000000000006 (SYMTAB)             0x2f0
 0x000000000000000a (STRSZ)              1 (bytes)
 0x000000000000000b (SYMENT)             24 (bytes)
 0x0000000000000015 (DEBUG)              0x0
 0x0000000000000007 (RELA)               0x310
 0x0000000000000008 (RELASZ)             31392 (bytes)
 0x0000000000000009 (RELAENT)            24 (bytes)
 0x000000006ffffffb (FLAGS_1)            Flags: PIE
 0x000000006ffffff9 (RELACOUNT)          1308
 0x0000000000000000 (NULL)               0x0
```

Рисунок 4 — Проверка заголовка и динамической секции blob-файла

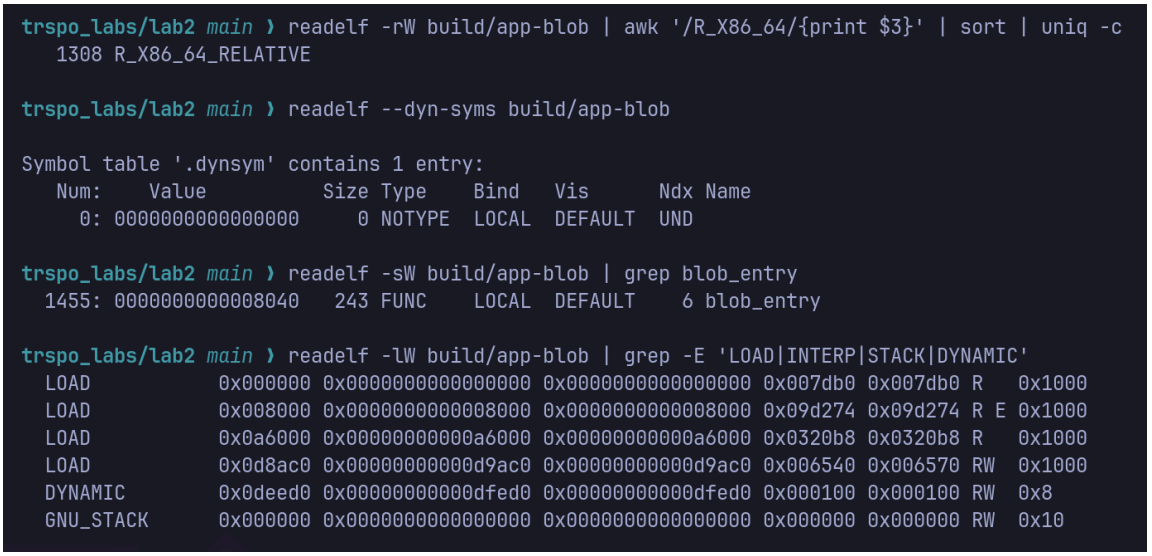


Рисунок 5 — Проверка релокаций, символов и сегментов blob-файла

3.6 Загрузчик ELF

build/elfloader — небольшая программа вокруг библиотеки elfload. Исходники библиотеки лежат в elfloader/elfload. Размер загрузчика — 23 208 байт.

Запуск:

./build/elfloader build/app-blob font.ttf out.png TRSPO

Загрузчик делает следующее:

1. Открывает blob-файл и читает его через pread.
2. Вызывает el_init. Библиотека проверяет, что это ELF нужной архитектуры, находит сегменты PT_LOAD и считает размер образа.
3. Выделяет память и вызывает el_load. Сегменты копируются в память, хвост BSS обнуляется.
4. Вызывает el_relocate. Применяются поправки R_X86_64_RELATIVE.
5. Считает адрес входа как базу плюс e_entry и вызывает blob_entry.

В аргументе передаётся структура blob_params: путь к шрифту, имя PNG, текст и заполненная таблица host_api. После возврата из blob загрузчик

освобождает память и закрывает файл. Если файл повреждён или это не ELF, загрузчик печатает ошибку el_init, el_load или el_relocate и не падает.

3.7 Корневой Makefile и инкрементная сборка

Корневой Makefile задаёт порядок. Сначала всегда собирается zlib: и libpng, и FreeType с ней линкуются.

Свои правила библиотек короче штатных configure. В zlib не входят файлы gz*.c: файловый gzip программе не нужен и тянет лишние вызовы libc. В libpng заголовок pnglibconf.h копируется из готового scripts/pnglibconf.h.prebuilt, свой configure не запускается. FreeType собирается по docs/INSTALL.ANY: один укрупнённый .c на модуль. Системный слой — src/base/ftsystem.c, он открывает шрифт через fopen. Флаг -DFT_CONFIG_OPTION_SYSTEM_ZLIB заставляет FreeType сжимать таблицы шрифта уже собранным zlib, а не своей копией.

Общие флаги библиотек: -O2 -fPIC. Защита стека и FORTIFY выключены. Они зовут функции glibc, которых в blob нет. Для FreeType ещё нужен -std=gnu11: при стандарте C23 glibc переименовывает strtol, а blob определяет обычное имя strtol.

3.8 Проверка результатов

Все три версии рисуют один и тот же PNG. Для текста TRSPO и шрифта font.ttf размер изображения 202×106 пикселей, размер файла 5669 байт. SHA-256 трёх файлов совпадает.

Таблица 2 — Размеры и характеристики исполняемых файлов

Файл	**Размер, байт**	**text**	**data**	**bss**
app-static	1006584	878532	26600	8
app-dynamic	21952	8658	712	168
app-blob	1004584	876853	25920	48

```
trspo_labs/lab2 main ❱ ./build/app-static font.ttf out-static.png TRSPO
load: 0.000000 s
zlib 1.2.11
total: 0.001023 s

trspo_labs/lab2 main ❱ ./build/app-dynamic font.ttf out-dynamic.png TRSPO
load: 0.003992 s
zlib 1.2.11
total: 0.004981 s

trspo_labs/lab2 main ❱ ./build/elfloader build/app-blob font.ttf out-blob.png TRSPO
load: 0.000000 s
zlib 1.2.11
total: 0.000998 s
```

Рисунок 6 — Вывод трёх версий программы

```
trspo_labs/lab2 main ❱ file out-static.png out-dynamic.png out-blob.png
out-static.png:  PNG image data, 202 x 106, 8-bit/color RGB, non-interlaced
out-dynamic.png: PNG image data, 202 x 106, 8-bit/color RGB, non-interlaced
out-blob.png:    PNG image data, 202 x 106, 8-bit/color RGB, non-interlaced

trspo_labs/lab2 main ❱ ls -l out-*.png
Permissions Size User    Date Modified Name
.rw-r--r--  5.7k drenk83  3 Nov 01:10  out-blob.png
.rw-r--r--  5.7k drenk83  3 Nov 01:10  out-dynamic.png
.rw-r--r--  5.7k drenk83  3 Nov 01:09  out-static.png

trspo_labs/lab2 main ❱ sha256sum out-*.png
e8bcbbeaba1db91e8da69b4b6409646eb042157bfe59648add11627d8b791cf0  out-blob.png
e8bcbbeaba1db91e8da69b4b6409646eb042157bfe59648add11627d8b791cf0  out-dynamic.png
e8bcbbeaba1db91e8da69b4b6409646eb042157bfe59648add11627d8b791cf0  out-static.png
```

Рисунок 7 — Проверка PNG-файлов

Ошибки не роняют программу. Во всех случаях печатается сообщение и возвращается код 0.

Таблица 3 — Проверка обработки ошибок

Ситуация	**Сообщение**	**Код возврата**
Нет аргументов	usage: ... font png text	0
Не хватает аргументов	usage: ... font png text	0

Несуществующий шрифт	cannot open font ...	0
Недоступный путь для PNG	cannot create png ...	0
Отсутствует libz.so.1	dlopen ... cannot open shared object file	0

Три версии собираются, запускаются и дают один PNG. Static не ищет три библиотеки при запуске. Dynamic открывает их явно и показывает время загрузки. Blob не имеет динамических зависимостей и запускается через elfload.

4 ВЫВОДЫ

Собрана программа, которая читает TTF, рисует строку в PNG через FreeType, libpng и zlib и печатает время загрузки и общее время работы.

Статическая версия включает libz.a, libpng.a и libfreetype.a в файл размером около 1,0 МБ. При запуске эти библиотеки не нужны, поэтому load около нуля. Динамическая версия занимает 22 КБ. Библиотек в ней нет: они открываются через dlopen в начале main, и это время видно отдельно от рисунка. Blob тоже около 1,0 МБ. В его динамической секции нет DT_NEEDED: код библиотек внутри файла, а функции libc приходят таблицей от загрузчика. Загрузчик на базе elfload копирует сегменты PT_LOAD, применяет 1311 поправку R_X86_64_RELATIVE и вызывает blob_entry.

Три версии дали один и тот же PNG. Сборка инкрементная: правка одного .c пересобирает связанный объектный файл и итоговый бинарник. Библиотеки описаны своими Makefile, это дополнительное задание 6.2.